

Day 5 GROUP BY And Joins

GROUP BY is used to group rows that have the same value in a particular column.

It is mainly used with aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN().

For example, if an Employee table contains employees from IT, HR, and Sales departments, GROUP BY Department will create separate groups for IT, HR, and Sales.

After grouping, we can calculate values for each group separately.

Example:

```
SELECT Department, AVG(Salary)
FROM Employee
GROUP BY Department;
```

This gives the average salary of each department.

Important points:

GROUP BY is generally used when we want department-wise, city-wise, product-wise, category-wise, or any similar grouped result.

Every normal column written in SELECT should usually also be present in GROUP BY.

WHERE is used to filter individual rows before grouping.

HAVING is used to filter groups after GROUP BY.

Example:

```
SELECT Department, COUNT()
FROM Employee
GROUP BY Department
HAVING COUNT() > 5;
```

This returns only those departments having more than 5 employees.

Order of execution is generally:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

JOINS — Notes

JOIN is used to combine data from two or more tables based on a common or related column.

For example, suppose one table contains employee information and another table contains department information.

Employee table may contain EmpID, Name, and DeptID.

Department table may contain DeptID and DepartmentName.

DeptID acts as the common column between the two tables.

INNER JOIN returns only those records that have matching values in both tables.

Example:

```
SELECT Employee.Name, Department.DepartmentName
FROM Employee
INNER JOIN Department
ON Employee.DeptID = Department.DeptID;
```

If an employee has a DeptID that does not exist in the Department table, that employee will not appear in an INNER JOIN.

LEFT JOIN returns all records from the left table and matching records from the right table.

If there is no match in the right table, SQL gives NULL for the right-table columns.

RIGHT JOIN returns all records from the right table and matching records from the left table.

If there is no matching record in the left table, SQL gives NULL for the left-table columns.

FULL OUTER JOIN returns all records from both tables.

It includes matching records as well as unmatched records from both tables.

CROSS JOIN creates every possible combination of rows from two tables.

For example, if the first table contains 3 rows and the second table contains 4 rows, CROSS JOIN returns 12 rows.

SELF JOIN means joining a table with itself.

It is useful when records inside the same table are related.

For example, an Employee table may contain EmployeeID and ManagerID. We can use a SELF JOIN to find the name of each employee's manager.

Easy way to remember:

GROUP BY = group similar rows and perform calculations.

JOIN = combine information from different tables.

```
CREATE TABLE sales (  
    id INT PRIMARY KEY,  
    salesperson VARCHAR(50),  
    city VARCHAR(50),  
    amount INT  
);
```

```
INSERT INTO sales VALUES  
(1, 'Aman', 'Delhi', 5000),  
(2, 'Aman', 'Delhi', 7000),  
(3, 'Riya', 'Mumbai', 4000),  
(4, 'Riya', 'Mumbai', 6000),  
(5, 'Riya', 'Mumbai', 8000),  
(6, 'Karan', 'Delhi', 3000),  
(7, 'Karan', 'Delhi', 2000),  
(8, 'Neha', 'Pune', 9000),  
(9, 'Neha', 'Pune', 7000),  
(10, 'Aman', 'Delhi', 6000);
```

Join

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT  
);
```

```
INSERT INTO employees VALUES  
(1, 'Aman', 101),  
(2, 'Riya', 102),  
(3, 'Karan', 101),  
(4, 'Neha', 103),  
(5, 'Rahul', NULL);
```

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
INSERT INTO departments VALUES  
(101, 'IT'),
```

```
(102, 'HR'),  
(103, 'Finance'),  
(104, 'Marketing');
```